

Tecnologías de BDD Distribuidas: NewSQL vs. NoSQL (YugabyteDB)

Ivan Andrés Villamarin

Aplicaciones Distribuidas

Facultad de Ciencias de la Computación

Universidad Técnica Estatal de Quevedo

7mo Software "A"

Resumen—YugabyteDB es una base de datos distribuida de tipo NewSQL que combina las propiedades transaccionales de los sistemas relacionales tradicionales con las capacidades de escalabilidad horizontal propias de los sistemas distribuidos. Este documento analiza sus características de distribución de datos, su modelo de consistencia según el teorema CAP, casos de uso documentados en la industria y su pertinencia para el proyecto SCLI (Sistema de Control de Laboratorios Informáticos).

Index Terms—NewSQL, NoSQL, YugabyteDB, bases de datos distribuidas, teorema CAP, consistencia, ACID.

I. INTRODUCCIÓN

YugabyteDB es una base de datos distribuida de tipo NewSQL, también conocida como Distributed SQL. Su objetivo principal es combinar las características tradicionales de las bases de datos relacionales, como el uso de SQL, transacciones ACID y consistencia fuerte, con capacidades propias de los sistemas distribuidos, como escalabilidad horizontal, alta disponibilidad y tolerancia a fallos [1], [5].

Esta tecnología surge como una alternativa para aplicaciones modernas que necesitan manejar grandes volúmenes de datos sin perder la confiabilidad de un modelo relacional. A diferencia de muchas bases NoSQL, que priorizan flexibilidad y escalabilidad con modelos de consistencia eventual, YugabyteDB mantiene un enfoque transaccional.

YugabyteDB permite trabajar con una API compatible con PostgreSQL llamada YSQL, lo cual facilita el uso de consultas SQL, relaciones entre tablas, claves primarias, índices y restricciones [5].

II. CARACTERÍSTICAS DE DISTRIBUCIÓN DE DATOS

YugabyteDB distribuye los datos mediante un mecanismo de fragmentación automática o *sharding*. Las tablas se dividen en partes más pequeñas llamadas *tablets*. Cada tablet contiene una porción de los datos y puede ubicarse en diferentes nodos del clúster [5].

Cada tablet se replica en varios nodos para garantizar disponibilidad y tolerancia a fallos. Para mantener la coordinación entre las réplicas, YugabyteDB utiliza el protocolo Raft. En este esquema, cada grupo de réplicas tiene un líder y varios seguidores [5].

Gracias a esta arquitectura, si un nodo falla, el sistema puede seguir funcionando usando las réplicas disponibles en otros nodos. Además, YugabyteDB permite desplegar bases

de datos en diferentes zonas o regiones, lo cual resulta útil para aplicaciones que necesitan continuidad operativa y baja latencia [5].

Otra característica importante es su compatibilidad con PostgreSQL. Esto permite que los desarrolladores usen herramientas, consultas y frameworks conocidos, reduciendo la dificultad de adopción [5].

III. MODELO DE CONSISTENCIA SEGÚN CAP

Según el teorema CAP, una base de datos distribuida debe tomar decisiones frente a tres propiedades: consistencia, disponibilidad y tolerancia a particiones. En escenarios de fallas de red, no siempre es posible garantizar las tres al mismo tiempo [1].

YugabyteDB se clasifica principalmente como una tecnología CP, es decir, prioriza la consistencia y la tolerancia a particiones. Esto significa que, ante una falla de comunicación entre nodos, el sistema busca mantener la información correcta y coherente, incluso si algunas operaciones deben esperar o limitarse temporalmente [3], [5].

Este modelo es importante en sistemas donde no se pueden aceptar datos contradictorios. Por ejemplo, en aplicaciones financieras, sistemas de inventario, plataformas de comercio electrónico o sistemas académicos, una inconsistencia podría causar errores importantes.

A diferencia de bases de datos NoSQL orientadas a disponibilidad y consistencia eventual, YugabyteDB conserva propiedades ACID. Esto permite ejecutar operaciones seguras, mantener restricciones y evitar conflictos en actualizaciones concurrentes [1], [5].

IV. CASO DE USO REAL DOCUMENTADO

Un caso real documentado de uso de YugabyteDB es el de Kroger, una empresa del sector minorista que adoptó esta tecnología para modernizar su plataforma de comercio electrónico. Según el caso publicado por YugabyteDB, Kroger utilizó esta base de datos distribuida para apoyar una arquitectura basada en microservicios, mejorar la disponibilidad y trabajar con una solución capaz de escalar en entornos modernos [6].

Otro caso documentado es Paramount+, plataforma de streaming que utilizó YugabyteDB dentro de una arquitectura multi-región para soportar eventos de alta demanda. En este tipo de escenario, la base de datos debe responder a grandes

cantidades de usuarios y mantener disponibilidad durante eventos críticos [7].

También se registra el caso de ComplyAdvantage, empresa dedicada a soluciones contra lavado de dinero y detección de fraude. Esta organización documentó el uso de YugabyteDB como parte de una plataforma distribuida para manejar servicios transaccionales, múltiples regiones y grandes volúmenes de datos [8].

Estos casos demuestran que YugabyteDB se utiliza en ambientes reales donde se requiere escalabilidad, consistencia, alta disponibilidad y soporte para arquitecturas modernas basadas en microservicios [6], [7], [8].

Para el proyecto SCLI — Sistema de Control de Laboratorios Informáticos, YugabyteDB resulta pertinente porque el sistema necesita administrar datos relacionales y transaccionales. El SCLI contempla módulos como usuarios, reservas, asistencia, inventario, laboratorios y reportes técnicos. Estos módulos se relacionan entre sí y requieren que la información se mantenga correcta.

Por ejemplo, el módulo de reservas necesita evitar que dos usuarios reserven el mismo laboratorio en el mismo horario. De igual manera, el inventario debe reflejar correctamente el estado de los equipos, ya sea disponible, en uso, dañado o en mantenimiento. En estos casos, la consistencia de los datos es más importante que aceptar operaciones conflictivas.

YugabyteDB se adapta bien al SCLI porque permite trabajar con SQL y con un modelo parecido a PostgreSQL, lo cual facilita su integración con un backend desarrollado en Spring Boot [5].

Desde el punto de vista del teorema CAP, para el SCLI conviene priorizar un modelo CP, es decir, consistencia y tolerancia a particiones. En este sistema es preferible limitar temporalmente una operación durante una falla antes que permitir reservas duplicadas, registros incorrectos de asistencia o inconsistencias en el inventario.

Por estas razones, YugabyteDB es una opción adecuada para el SCLI, ya que permite justificar técnicamente una base de datos distribuida moderna, consistente, escalable y compatible con aplicaciones relacionales.

IV-A. Resumen de parámetros de importancia

V. CONCLUSIÓN

En resolución, YugabyteDB es una base de datos distribuida adecuada para sistemas modernos porque permite manejar información en varios nodos sin perder la consistencia de los datos. Su funcionamiento combina las ventajas de SQL, la escalabilidad horizontal y la alta disponibilidad, lo que la convierte en una alternativa sólida frente a bases de datos tradicionales y algunas soluciones NoSQL. Además, al priorizar la consistencia, ayuda a evitar errores como datos duplicados o registros contradictorios. Por ello, su uso resulta apropiado en aplicaciones donde la información debe mantenerse correcta, segura y disponible.

VI. REPOSITORIO

<https://github.com/iavillamarin98-pred/FORM-IC-02-Investigaci-n-Individual-Tecnolog-as-de-BDD-Distribuidas>

REFERENCIAS

- [1] N. Chaudhry and M. M. Yousaf, "Architectural assessment of NoSQL and NewSQL systems," *Distributed and Parallel Databases*, vol. 38, no. 4, pp. 881–926, Dec. 2020, doi: 10.1007/s10619-020-07310-1.
- [2] Y. Watanabe, R. Kawashima, and H. Matsuo, "Proxy-based Transaction Acceleration for NewSQL," in *2023 Eleventh International Symposium on Computing and Networking Workshops (CANDARW)*, Matsue, Japan, 2023, pp. 335–341, doi: 10.1109/CANDARW60564.2023.00065.
- [3] Z. Zhang, A. Megargel, and L. Jiang, "Performance Evaluation of NewSQL Databases in a Distributed Architecture," *IEEE Access*, vol. 13, pp. 11185–11194, 2025, doi: 10.1109/ACCESS.2025.3529740.
- [4] E. Pina, F. Sá, and J. Bernardino, "NewSQL Databases Assessment: CockroachDB, MariaDB Xpand, and VoltDB," *Future Internet*, vol. 15, no. 1, p. 10, Jan. 2023, doi: 10.3390/fi15010010.
- [5] YugabyteDB, "YugabyteDB Architecture," YugabyteDB Docs, 2025. [Online]. Available: <https://docs.yugabyte.com/stable/architecture/>
- [6] YugabyteDB, "Kroger Embraces Distributed SQL Database with YugabyteDB for Ecommerce," YugabyteDB Customer Success Stories, 2024. [Online]. Available: <https://www.yugabyte.com/success-stories/kroger-embraces-distributed-sql-database-with-yugabytedb-for-ecommerce/>
- [7] YugabyteDB, "Touchdown! Paramount+'s Record-Setting Big Game Live Stream with YugabyteDB," YugabyteDB Blog, 2024. [Online]. Available: <https://www.yugabyte.com/blog/super-bowl-stream-paramount-plus-architecture/>
- [8] ComplyAdvantage, "Lessons from Building AML and Fraud Detection with Distributed SQL," ComplyAdvantage Technology Blog, 2025. [Online]. Available: <https://technology.complyadvantage.com/lessons-from-building-aml-and-fraud-detection-with-distributed-sql---dss-2024/>

Cuadro I
PARÁMETROS CLAVE DE YUGABYTEDB

Aspecto	Resumen	Importancia
Tipo de base de datos	YugabyteDB es una base de datos NewSQL o Distributed SQL.	Combina SQL tradicional con capacidades distribuidas modernas.
Distribución de datos	Divide los datos en tablets y los reparte entre varios nodos.	Permite escalar el sistema agregando más servidores.
Replicación	Copia los datos en varios nodos mediante el protocolo Raft.	Ayuda a mantener el sistema disponible aunque falle un nodo.
Consistencia	Prioriza la consistencia fuerte y las transacciones ACID.	Evita datos duplicados, contradictorios o incorrectos.
Modelo CAP	Se aproxima al modelo CP: consistencia y tolerancia a particiones.	Es útil en sistemas donde la exactitud de la información es más importante que aceptar operaciones inconsistentes.
Compatibilidad	Usa YSQL, compatible con PostgreSQL.	Facilita la integración con herramientas conocidas y aplicaciones desarrolladas con Spring Boot.
Escalabilidad	Permite crecimiento horizontal al agregar nuevos nodos.	Es adecuada para sistemas que pueden aumentar usuarios, datos o servicios.
Casos reales	Ha sido documentada en escenarios como Kroger, Paramount+ y ComplyAdvantage.	Demuestra que se usa en aplicaciones reales de comercio electrónico, streaming y servicios transaccionales.
Pertinencia general	Es útil para aplicaciones que requieren datos correctos, disponibilidad y crecimiento.	Representa una alternativa sólida frente a bases tradicionales centralizadas y algunas NoSQL.